

Methodology Report: E-Commerce Conversion Prediction Challenge

Event: Summer Analytics 2026 Mini-Hackathon (Week 2)

Deliverable: 3 (One-Page Summary)

1. Overview & Objective

The objective of this challenge was to develop a robust machine learning model capable of predicting customer conversion (1 = Converted, 0 = Not Converted) based on structured, anonymized e-commerce tabular data. Given the business context, identifying true conversions accurately without excessive false positives is critical; therefore, the model architecture and preprocessing steps were specifically optimized for the **F1-Score**, ensuring a strict balance between precision and recall.

2. Exploratory Data Analysis & Preprocessing

An initial analysis of the datasets (train.csv, public_test.csv, private_test.csv) revealed a mix of continuous numerical features (e.g., Age, Income, Time_On_Site) and categorical features (e.g., City_Tier, Device_Type, Traffic_Source). Notably, the dataset contained missing values across several key features, and the target variable (Converted) exhibited standard e-commerce class imbalance.

To ensure data integrity and prevent data leakage, a strict Scikit-Learn ColumnTransformer pipeline was engineered:

- **Numerical Features:** Missing continuous values were handled using a SimpleImputer with a **median strategy**. The median was selected over the mean to provide robustness against extreme outliers in features like Income and Time_On_Site. Post-imputation, the data was standardized using StandardScaler to ensure uniform feature contribution during model optimization.
- **Categorical Features:** Missing categorical data was imputed using the **most frequent** (mode) strategy. The features were then transformed via OneHotEncoder. To ensure the pipeline did not fail on the private test set, the handle_unknown='ignore' parameter was implemented, safely neutralizing any unseen categorical values.

3. Model Selection & Architecture

A **Random Forest Classifier** was selected as the primary predictive model. Random Forest was chosen over baseline linear models (such as Logistic Regression) because it naturally captures complex, non-linear relationships between browsing behaviors (e.g., Pages_Viewed and Time_On_Site) and conversion outcomes without requiring exhaustive manual feature crossing.

To directly address the target variable imbalance, the model was instantiated with the class_weight='balanced' parameter. This technique algorithmically adjusts the weights inversely proportional to the class frequencies in the input data, heavily penalizing the model for misclassifying the minority class (conversions) and significantly boosting the recall metric.

4. Training, Validation & Evaluation

During the development phase, the model was validated using an 80/20 train_test_split. The primary evaluation metric was the F1-Score, supplemented by the overall classification report to monitor precision and recall trade-offs. The Random Forest hyperparameter max_depth was constrained to 10 to prevent overfitting to the training data, allowing the model to generalize effectively to unseen instances.

5. Final Prediction Generation

For the final submission, the validated pipeline (encapsulating both the preprocessing transformer and the Random Forest classifier) was retrained on the *entire* train.csv dataset to maximize its learning context. Predictions were then generated strictly on the isolated features of private_test.csv. The resulting binary outputs were concatenated alongside the respective User_ID array to generate the final submission.csv in strict accordance with the hackathon's formatting requirements. All code is completely reproducible via the provided Jupyter Notebook.